

深入分析区间型动态规划

郑州市第九中学 张旭祥

区间型动态规划在信息学竞赛中应用甚广，它是动态规划中的经典问题，最小代价字母树是这类动态规划最经典的体现，对于初学者而言这类动态规划并不太好理解。于是，区间型动态规划又成了动态规划中的难点问题。下面，本文将为大家深入分析区间型动态规划的思想及其程序的实现，希望能对大家有所帮助。

- 历届大赛中区间型动态规划题目的考查。

区间型动态规划是各大信息竞赛出题的热点，具体体现在以下题目：

1. 合并石子----NOI1995
2. 能量项链---NOIP2006
3. 加分二叉树----NOIP2003
4. 最优排序二叉树---- CTSC96

这些题目出现的频次及其所在比赛的重要性足以说明区间型动态规划在各类动态规划中有着举足轻重的地位。

二、区间型动态规划的算法分析

在这里就以经典的最小代价字母树作为例子，对区间型动态规划的算法进行分析。

问题描述：

给定一个序列，如 4、1、2、3 我们将他们相加进行合并，最终合并成一个数，每次相加的代价是两个加数的和，求怎样的相加顺序可以使总代价最小。

很多初学者认为这类动态规划不易理解，其重要原因是这类动态规划与其他动态规划的思想不大相同，而初学者又是利用其他动态规划的思想来解决这类动态规划，从而进入了思维误区。这种错误的思维模式一旦建立便很难重新建立正确的解题思想，从而陷入绝境。

这类动态规划正确的解法是这样的：

首先，根据动态规划无后效性的性质可以想到：对于一个序列：A1、A2.....An，假如最后相加的两个数是第一个数到第 i 个数的和 $S[1--i]$ 以及第 i+1 个数到第 n 个数的和 $S[i+1--n]$ ，另外，对于第一个数到第 i 个数相加的最小代价是 $F[1,i]$ 以及从第 i+1 到第 n 个数相加的最小代价为 $F[i+1,n]$ ，则总代价即为 $F[i+1,n]+F[1,i]$ (前面相加的最小代价) + $S[1--i] + S[i+1--n]$ (最后一次相加的最小代价)。由此，我们可以清楚的看出要想求出总代价的最小值只要枚举 i 的位置，使得 $F[i+1,n]+F[1,i] + S[1--i] + S[i+1--n]$ 的和最小即可。综上所述我们可以总结出状态转移方程：

$F[i,j] := \min\{F[i,k] + F[k+1,j] + S[i,k] + S[k+1,j]\}$

状态转移数组 F 即代表从第 i 个数到第 j 个数相加的最小代价, S 数组为预处理好的从第 i 个数到第 j 个数的和。

核心代码如下:

For $i:=1$ to n do

For $j:=1$ to $n-1$ do

For $k:=j$ to $i+j-1$ do

If $F[j,i+j] > F[j,k] + F[k+1,i+j] + S[j,k] + S[k+1,i+j]$

then $F[j,i+j] := F[j,k] + F[k+1,i+j] + S[j,k] + S[k+1,i+j];$

最小值 ANS 为 $F[1,n]$.

二、区间型动态规划的具体应用

1、石子归并

题目描述:

在一个圆形操场的四周摆放着 N 堆石子($N \leq 100$),现要将石子有次序地合并成一堆.规定每次只能选取相邻的两堆合并成新的一堆,并将新的一堆的石子数,记为该次合并的得分.编一程序,由文件读入堆栈数 N 及每堆栈的石子数(≤ 20).

(1)选择一种合并石子的方案,使用权得做 $N-1$ 次合并,得分的总和最小;

(2)选择一种合并石子的方案,使用权得做 $N-1$ 次合并,得分的总和最大;

输入数据:

第一行为石子堆数 N ;

第二行为每堆的石子数,每两个数之间用一个空格分隔.

输出数据:

从第一至第 N 行为得分最小的合并方案.第 $N+1$ 行是空行.从第 $N+2$ 行到第 $2N+1$ 行是得分最大合并方案.每种合并方案用 N 行表示,其中第 i 行($1 \leq i \leq N$)表示第 i 次合并前各堆的石子数(依顺时针次序输出,哪一堆先输出均可).要求将待合并的两堆石子数以相应的负数表示.

输入输出范例:

输入:

4

4 5 9 4

输出:

-4 5 9 -4

-8 -5 9

-13 -9

22

4 -5 -9 4

4 -14 -4

-4 -18

22

这道题目可以说跟最小代价字母树只有两个不同的地方，一个是所求序列是一个环形的，另一个是要求输出方案，对于输出方案而言，只需要用一般动态规划记录方案的方法即可，因为不是本文的重点在此就不再深究。对于所求序列是环形的问题其实只需要用一个小小的技巧便轻松解决，请先看代码：

```
{-----预处理-----}
```

```
Read(n);
```

```
For i:=1 to n do
```

```
Begin
```

```
    Read(a[i]);
```

```
    a[i+n]:=a[i];
```

```
End;
```

```
For i:=1 to n do
```

```
For j:=1 to n do
```

```

For k:=i to j do

    S[l,j]:=S[l,j]+a[k];

{ -----DP 过程----- }

For i:=1 to n do

For j:=1 to 2*n-l do

    For k:=j to i+j-1 do

If F[j,i+j]>F[j,k]+F[k+1,i+j]+S[j,k]+S[k+1,i+j]

    then F[j,i+j]:=F[j,k]+F[k+1,i+j]+S[j,k]+S[k+1,i+j];

For i:=1 to n-1 do

    Ans:=min{F[i,i+n-1]}

```

最小值为 Ans

从代码中可以看出这道题的写法跟最小代价字母树的区别在于枚举起点的时候长度增加到了 $2*n$ ，并且在最后求解的时候也需要枚举起点，求长度为 n 的最小值，这恰恰是利用了区间型动态规划的特点。当然，在读入数据的时候需要把初始数组的长度扩大一倍然后再进行预处理即可。这种方法在能量项链一题中还有具体的体现，因为能量项链的核心算法与本题几乎一样，所以就不再赘述。大家可以自己练习。

2、加分二叉树

【问题描述】

设一个 n 个节点的二叉树 **tree** 的中序遍历为 $(1,2,3,...,n)$ ，其中数字 $1,2,3,...,n$ 为节点编号。每个节点都有一个分数（均为正整数），记第 j 个节点的分数为 d_i ，**tree** 及它的每个子树都有一个加分，任一棵子树 **subtree**（也包含 **tree** 本身）的加分计算方法如下：

subtree 的左子树的加分 $\times$ **subtree** 的右子树的加分 $+$ **subtree** 的根节点的分数

若某个子树为空，规定其加分为 1，叶子的加分就是叶节点本身的分数。不考虑它的空子树。

试求一棵符合中序遍历为 $(1,2,3,...,n)$ 且加分最高的二叉树 **tree**。要求输出：

(1) **tree** 的最高加分

(2) **tree** 的前序遍历

【输入格式】

第 1 行：一个整数 n ($n < 30$)，为节点个数。

第 2 行： n 个用空格隔开的整数，为每个节点的分数（分数 < 100 ）。

【输出格式】

第 1 行：一个整数，为最高加分（结果不会超过 4,000,000,000）。

第 2 行： n 个用空格隔开的整数，为该树的前序遍历。

【输入样例】

5

5 7 1 2 10

【输出样例】

145

3 1 2 4 5

这道题目巧妙地将区间型动态规划和二叉树相结合，既考查了二叉树的基本性质又考查了大家对动态规划的掌握，不得不承认这是一道经典好题。同样，这道题最后要求输出前序遍历，只需要用递归建树即可，这里就不多说了。具体的预处理过程和动态规划过程如下：

{-----预处理-----}

read(n);

For i:=1 to n do

read(a[i]);

For i:=0 to n do

For j:=0 to n do

F[i,j]:=1;

For i:=1 to n do

F[i,i]:=a[i];

{ -----DP----- }

For i:=2 to n do

For j:=1 to n-i+1 do

For k:=j to i+j-1 do

if $f[j, i+j-1] < f[j, k-1] * f[k+1, i+j-1] + a[k]$

then $f[j, i+j-1] := f[j, k-1] * f[k+1, i+j-1] + a[k];$

其中 $A[k]$ 是读入的数组， $F[1, n]$ 同样为最终结果，表示从第一个到第 n 个数进行建树的最大价值。

小结：对于区间型动态规划的和具体的应用就是这些，其实这类动态规划并不难，关键在于领悟区间的含义，更重要的是将这种思想进行变通，灵活应用。另外，在程序的实现过程中掌握一些技巧也是必须的，这是轻松解题的关键，最后后希望大家能够通过此文轻松掌握区间型动态规划。